Developer Manual

Student Code Online Review and Evaluation (2.0)

Student Code Online Review and Evaluation (2.0) or S.C.O.R.E (2.0) is a platform for creating and submitting programming assignments.

Team Members

Dorothy Ammons - dammons2022@my.fit.edu
Patrick Kelly - pkelly2022@my.fit.edu
Shamik Bera - sbera2022@my.fit.edu
Rak Alsharif - ralsharif2021@my.fit.edu

Faculty Advisor and Client

Raghuveer Mohan

April 19th, 2026

Table of Contents

Table of Contents - 1

1. Introduction - 3

    1.1 About S.C.O.R.E (2.0)

    1.2 Requirements

    1.3 Google OAuth Data Collection

2. System Architecture - 4

    2.1 Frontend Framework

    2.2 Backend Framework

    2.3 Google OAuth

    2.4 Database Layer

    2.5 Example Flow

3. File Organization - 7

    3.1 GitHub Repository

    3.2 File Structure

    3.3 Backend Files

    3.4 Frontend Files

4. Automatic Grading System - 9

    4.1 Overview

    4.2 Input Structure

    4.3 Program Execution

4.4 Language Support

5. User Roles and Permissions - 11

5.1 Student Role

5.2 Professor Role

5.3 Role Enforcements

6. Database - 13

6.1 Firebase

6.2 Google Cloud Service Accounts

7. Building the Application - 16

7.1 React Build Process

7.2 PyInstaller

Introduction

1.1 About S.C.O.R.E (2.0)

Student Code Online Review and Evaluation (2.0) or S.C.O.R.E (2.0) is a platform for creating and submitting programming assignments. Professors are able to create classes, assignments, rubrics, test cases and rosters. Additionally, they may view grades, submissions, AI usage scores and similarity scores. Students may use the application to view and make submissions to their assignments, receiving automatic grades and feedback through their submission output.

1.2 Requirements

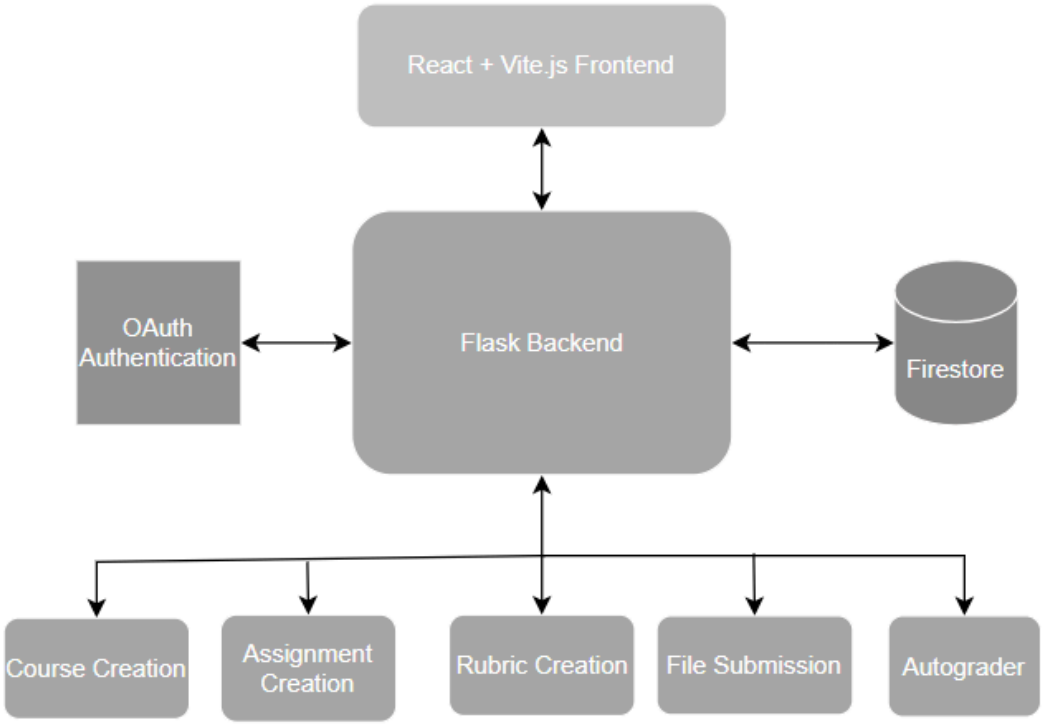
In order to develop S.C.O.R.E (2.0) as expected, the following are required:

- Internet access
- Windows or Linux operating system
- Google Chrome installed with the desired S.C.O.R.E (2.0) login email
- S.C.O.R.E (2.0) executable
- GitHub
- Projectscore2.0@gmail.com login
- npm
- Python
- PyInstaller
- scikit-learn
- PyTorch

1.3 Google OAuth Data Collection

Our platform uses Google OAuth as a verification and identification tool. We store email addresses to remember and identify users. We do not collect or store any other personal data. We do not sell user data to third parties.

System Architecture



S.C.O.R.E (2.0) follows a client-server architecture. It uses a React framework for the frontend, a Flask backend, and Firestore for the cloud database. Authentication is handled though Google OAuth.

2.1 Frontend Framework

The frontend is built using React with Vite. By using REST APIs, the frontend

- Renders UI components
- Handles user interactions
- Sends API requests to the backend
- Displays responses

2.2 Backend Framework

The backend is built using Flask. The backend responsibilities are to

- Expose API endpoints
- Handle logic
- Control database operations
- Connect to OAuth authentication

Some important endpoints are

- User creation: create and validate users
- Class creation: create a class and store related data
- Assignment creation: create assignments and store related data
- File submission: accept and parse file uploads from users
- Autograder: run and evaluate student submissions

2.3 Google OAuth

User authentication is handled with Google OAuth. The flow is as follows:

1. User starts logging in from frontend
2. Google OAuth redirects
3. Google OAuth returns an access token
4. Backend validates token
5. Session is created

2.4 Database Layer

Data is stored in Google Firestore. This cloud database allows users to run their own copy of the application locally and still connect to every other running instance. Firestore is in charge of

- Storing users, classes, assignments, submissions, and grades
- Providing real updates
- Supporting collections and documents

2.5 Example Flow

1. User runs the executable
2. Flask starts running and serves the frontend
3. The frontend loads the login page

4. User clicks sign in
5. Frontend registers response and initiates authentication
6. Google OAuth window opens
7. User completes authentication
8. Frontend sends token to backend
9. Backend verifies the user and sends the frontend session data
10. Frontend loads the user’s dashboard

File Organization

3.1 GitHub Repository

The GitHub repository is connected to the projectscore20 GitHub account. The following command will allow you to fork your own copy of the repository.

git clone https://github.com/projectscore20/Automated-Grading-System-SCORE-2.0.git

You may then navigate to the directory.

```
cd Automated-Grading-System-SCORE-2.0
```

You now have the S.C.O.R.E (2.0) files locally.

Additionally, the webpage that has the loading page is under the projectscore20.github.io repository.

3.2 File Structure

The main directories that you would need are the

- Backend
  - API endpoints
  - Auto test logic
  - Similarity detection logic
- ai_detection
  - AI detectors
  - AI code detection logic
- autoTest
  - Auto feedback and grading logic
  - Auto test logic
- web_app
  - Vite configuration
  - Src files
    - Routes
    - All frontend files
- dist
  - Frontend build

3.3 Backend Files

Your main backend file is going to be app.py. This file contains just about every endpoint for every route. It controls Flask, threads, and database connections.

Your auto test files will be in the autoTest folder. This includes auto_feedback_object.py, which compares output, applies grades and stores results. The other important file is auto_test_object.py. This file compiles the code, runs the code, captures errors and runtime, and stores output. Supported languages are Python, Go, C, C++, Haskell, Rust, and Java.

Any changes you make to these files should be copied to the same autoTest file in the original directory.

Lastly, your similarity score logic is in the cops.py file. This is the file that requires scikit-learn and compares similarities between submissions.

3.4 Frontend Files

Your frontend files will be in the web_app folder. This folder contains your index.html, vite configuration, and src folder. In the src folder, you will see your components, assets, routes and a collection of jsx and css files. These are what create the visuals and interactions of the web application.

Automatic Grading System

4.1 Overview

The auto-grader follows a structured pipeline:

1. Load test cases
2. Validate submission file
3. Compile code (if required)
4. Execute code against each test case
5. Capture output, errors, and runtime
6. Store results in a structured JSON format

4.2 Input Structure

The auto-grader accepts the

- Submission file
- Code language
- Test cases
    - Identifier
    - Input
    - Output

4.3 Program Execution

The run function

- Executes shell commands using subprocess
- Passes input into the code
- Saves
    - Output
    - Errors
    - Execution time
- Enforces a timeout of 60 seconds

4.4 Language Support

The automatic grading system supports a variety of languages. It is crucial that the student submitting the program has the required libraries installed locally. The supported languages are as follows:

- Interpreted Languages
    - Python
    - Go
- Compiler Languages
    - C
    - C++
    - Haskell
    - Rust
    - Java

User Roles and Permissions

5.1 Student Role

Students are the primary users who will interact with the system. Their permissions are limited to prevent unauthorized changes to class content.

Students are able to

- View registered classes
- View assignment details and visible test cases
- Upload and submit code files
- Receive automated grades and feedback
- View submission history

Students are not able to

- Create or modify classes
- Create or modify assignments
- Access other user's data
- Modify grades

5.2 Professor Role

Professors have control over class and assignment operations. Their permissions allow them to create and evaluate the different structures.

Professors are able to:

- Create classes
- Add students to class rosters
- Create assignments with descriptions and deadlines
- Create test cases and feedback
- Create rubrics
- View all student submissions
- View grades
- Generate similarity score heatmaps
- View AI usage detection results

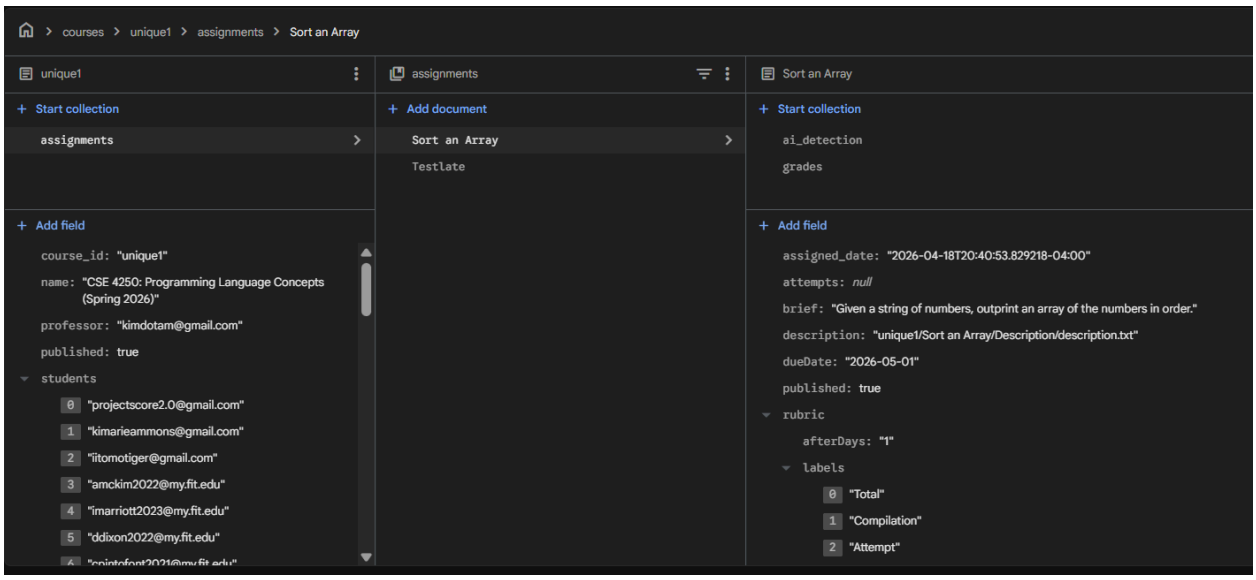
5.3 Role Enforcements

Roles are decided at user creation. The backend ensures user type and permissions before interacting with the frontend. This creates student sided views and professor style views. Additionally, professors may not view or interact with any classes they have not created and students may not view or interact with any classes they have not been registered for.

Database

6.1 Firebase

Firebase is the cloud database we use for the application. To reach the console, go to console.firebase.google.com, login with the projectscore2.0 email and click on Score20. The ID is score20-a5ae0. On the left panel you can navigate to Firestore. This is where you will see all stored files related to users, classes, and assignments. The view will look similar to this:

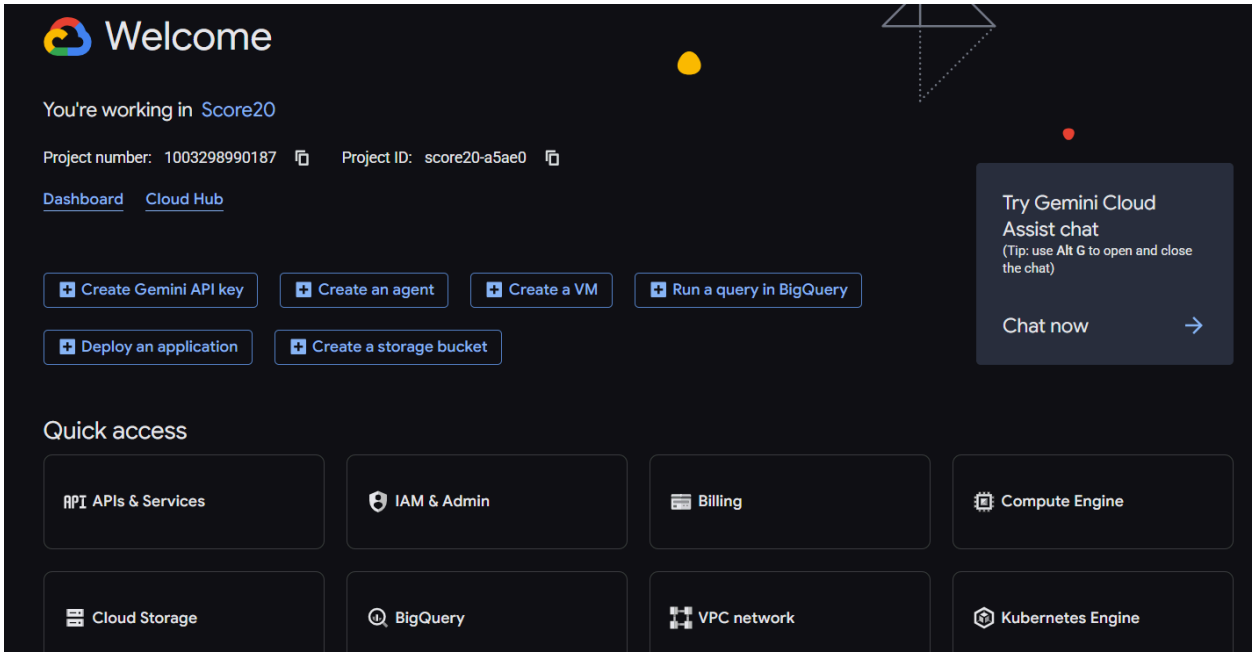


Right below it will be the Storage shortcut. Here you will see larger files stored including, descriptions and submissions.

Firestore allows for 20k writes, 50k reads, and 20k deletes per day on the free tier. Storage is through Google Cloud Firestore which allows for up to 1 GiB in total free.

6.2 Google Cloud Service Accounts

To reach the Google Cloud Console navigate to cloud.google.com and login with the projectscore2.0 email. Next, click the “Console” button in the top right. You should see the Score20 console with the same ID as the Firestore console.



To reach the service accounts, navigate to APIs and Services, then on the left click Credentials. Here you will see your browser key, OAuth Client ID, and service accounts. The OAuth Client ID is what allows the application to use Google OAuth for logins.

Building the Application

7.1 React Build Process

To begin building your application for use, first create the dist file. In your terminal, go to your project folder. Then, proceed into the web_app folder. Here you will run the following command:

```
npm run build
```

Once this process is complete, open the web_app folder in your file management application. Copy the new folder titled “dist” into your main directory and Backend folders. You will have to do this every time you make changes to any of your frontend files.

7.2 PyInstaller

To wrap your files into an executable, use PyInstaller. From your main directory in your terminal run:

```
python -m PyInstaller --onefile --noconsole --icon=SCORE2.0Logo.ico --add-data "dist;dist" --add-data "Backend/score20-a5ae0-633d295581e6.json;Backend" --add-data "ai_detection;ai_detection" --hidden-import=cops --exclude-module torch.utils.tensorboard --exclude-module tensorboard Backend/app.py
```

Alternatively, on Linux machines run:

```
python -m PyInstaller --onefile --add-data "dist:dist" --add-data "Backend/score20-a5ae0-633d295581e6.json:Backend" --add-data "ai_detection:ai_detection" --hidden-import=cops --exclude-module torch.utils.tensorboard --exclude-module tensorboard Backend/app.py
```

With this command, you may have to use venv. If you receive errors, from your project root try running:

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install pyinstaller
```

Then continue with the PyInstaller command.

If you wish to build the Linux file on a Windows operating system, simply use

```
wsl -d Ubuntu
```

and continue with the previous commands.

Once the build is complete, your application will be located in the dist folder in your main directory. You may rename the file to S.C.O.R.E or similar name.